

Ghid complet pentru dezvoltare DevSecOps open source

Calitatea codului securitate conformitate review si validare obligatorie a deploymentului

Ediția 1.0 | 12 septembrie 2026 | ERNU.EU

Acest manual explică implementarea de la zero a unui flux DevSecOps reproductibil într-un proiect GitHub nou. Recomandarea este să începeți cu audit manual și neblocant, să calibrați constatările, apoi să transformați controlul într-un gate obligatoriu pentru merge, release și deployment.

Public țintă: dezvoltatori, administratori GitHub, DevOps, responsabili de securitate și manageri tehnici.

Profil	Trigger	Efect
manual-nonblocking	workflow_dispatch	numai rapoarte
pr-advisory	pull request	adnotări
mandatory-deployment-gate	PR / release	blochează deploymentul

Cuprins

1. Decizia recomandată
2. Arhitectura și responsabilitățile
3. Pregătirea proiectului GitHub
4. Instalarea SonarQube Community
5. Instalarea și configurarea scanerelor
6. Rularea manuală neblokantă
7. Interpretarea rezultatelor și excepțiile
8. Trecerea la validare obligatorie
9. Protejarea deploymentului
10. ORT pentru compliance și SBOM
11. PR Agent cu Ollama
12. Operare mentenanță și troubleshooting
13. Checklist pentru proiect nou
14. Anexe cu fișiere complete

Decizia recomandată

Stackul recomandat

SonarQube Community, Semgrep CE, MegaLinter, Trivy, Gitleaks și Reviewdog formează baza potrivită. ORT completează controlul licențelor și inventarul software, iar PR-Agent cu Ollama oferă review AI self-hosted. Niciun instrument nu acoperă singur calitatea, SAST, vulnerabilitățile, secretele, infrastructura ca cod și licențele.

Principiul de implementare

Profilul inițial manual-nonblocking rulează numai la cerere și produce artefacte. După două până la patru săptămâni de baseline, regulile stabile trec în profilul mandatory-deployment-gate. Denumirea jobului devsecops-gate rămâne neschimbată pentru Branch Protection.

Arhitectura și responsabilitățile

Componente și roluri

SonarQube gestionează quality gate; Semgrep identifică modele nesigure; MegaLinter normalizează conformitatea; Trivy verifică CVE, IaC, filesystem și imagini; Gitleaks detectează secrete; Reviewdog adnotează diff-ul; ORT evaluează licențe; Ollama păstrează review-ul AI în infrastructura controlată.

Control	Tool	Decision
Quality	SonarQube	Quality Gate
SAST	Semgrep	Rule severity
Compliance	MegaLinter	Lint policy
Supply chain	Trivy and ORT	CVE license SBOM
Secrets	Gitleaks	Any confirmed secret
Review	Reviewdog and PR-Agent	Human approval

Model de încredere

Workflow-ul primește permisiunea minimă contents read. Security events write se acordă numai când se publică SARIF. Tokenurile rămân în GitHub Secrets sau într-un secret manager. Runnerul self-hosted pentru SonarQube și Ollama nu trebuie expus direct internetului.

Pregătirea proiectului GitHub

Cerințe

Aveți nevoie de un repository GitHub, Actions activ, Docker Engine și Compose v2. Pentru SonarQube sunt recomandate cel puțin 4 vCPU, 8 GiB RAM, PostgreSQL persistent, DNS, TLS și backup testat. Se separă runnerul de analiză de gazda de producție.

Area	Minimum	Production recommendation
Runner	Linux and Docker	Ephemeral or dedicated
SonarQube	4 vCPU 8 GiB	Capacity monitored
Database	PostgreSQL volume	Encrypted tested backup
Network	DNS and TLS	Private access and reverse proxy

Structura repository-ului

Copiați configurațiile în devsecops/config, scripturile în devsecops/scripts și workflow-urile active în .github/workflows. Exemplele cu extensia .example rămân inactive. Rapoartele generate se exclud din Git.

Instalarea SonarQube Community

Instalarea platformei

Copiați platform/sonarqube/.env.example ca .env și înlocuiți toate valorile CHANGE_ME. Verificați docker compose config, porniți serviciile și publicați numai interfața SonarQube prin HTTPS. PostgreSQL nu se expune. Schimbați parola inițială și creați un token dedicat proiectului.

Integrarea GitHub

Creați SONAR_HOST_URL și SONAR_TOKEN în Actions Secrets. Runnerul care execută scannerul trebuie să rezolve DNS-ul și să acceseze URL-ul SonarQube. Nu includeți adrese sau tokenuri reale în workflow ori rapoarte.

Instalarea și configurarea scanerelor

Semgrep MegaLinter Trivy și Gitleaks

Păstrați politici versionate. În pilot, comenzile produc SARIF și rapoarte chiar dacă există constatări. Pentru profilul blocant, eliminați exit-code 0 și continue-on-error, fixați versiunile imaginilor și activați numai reguli calibrate.

Reviewdog

Reviewdog publică observații pe liniile modificate și reduce zgomotul în pull request. Permisia pull-requests write se acordă numai workflow-ului care adnotează. Rezultatul scannerului continuă să fie sursa tehnică autoritativă.

Rularea manuală neblokantă

Procedura manuală

Executați validatorul configurațiilor, apoi scriptul local sau Actions Run workflow. Descărcați artefactele, eliminați duplicatele, confirmați constatările, desemnați proprietar și termen, remediați și reexecutați. Auditul manual nu trebuie să pornească deployment.

Clasificarea

Secretele confirmate și vulnerabilitățile critice au prioritate imediată. Constatările false pozitive se suprimă cât mai îngust. Fiecare excepție conține identificator, componentă, justificare, control compensator, proprietar și expirare.

Interpretarea rezultatelor și excepțiile

Calibrarea gate-ului

Măsurați timpul de rulare, stabilitatea, rata fals pozitivă și backlogul. Nu activați blocarea dacă scannerul nu poate rula repetabil. Începeți cu secrete confirmate și CVE critice, apoi adăugați severitatea HIGH și regulile de calitate convenite.

Branch Protection

Activați Require a pull request before merging și Require status checks to pass. Selectați exact devsecops-gate. Dezactivați bypass pentru contributorii obișnuiți, cereți ramură actualizată și protejați ștergerea sau force-push-ul.

Setting	Value	Purpose
Pull request	Required	No direct merge
Status check	devsecops-gate	Stable contract
Bypass	Disabled	Consistent enforcement
Environment	Required reviewers	Production approval

Trecerea la validare obligatorie

Contractul de deployment

Joburile build, package și deploy declară needs pentru devsecops-gate. Deploymentul rulează numai când rezultatul este success. Se scanează artefactul sau imaginea finală după build, iar Production folosește Environment Protection și required reviewers.

Testul negativ obligatoriu

Introduceți într-un pull request de test o mostră inofensivă detectată de o regulă. Confirmați că gate-ul eșuează, merge-ul nu poate fi executat și deploymentul nu pornește. Eliminați mostra și confirmați că același flux devine verde.

Protejarea deploymentului

ORT

ORT inventariază pachetele, licențele și proveniența, apoi evaluează politica organizației. Publicați rezultatele și SBOM-ul ca artefacte. O licență necunoscută necesită review; o licență interzisă blochează profilul obligatoriu.

PR Agent și Ollama

Rulați Ollama pe o gazdă internă și limitați accesul runnerului. PR-Agent trebuie configurat consultativ la început. Nu trimiteți secrete sau cod restricționat către servicii externe. Review-ul AI nu înlocuiește aprobarea umană.

ORT pentru compliance și SBOM

Mentenanță

Actualizați lunar imaginile și acțiunile într-un pull request dedicat. Testați backupul și restaurarea SonarQube/PostgreSQL. Monitorizați durata, erorile infrastructurii și expirarea excepțiilor. Investigați separat eșecul scannerului de constatarea de securitate.

Troubleshooting

Dacă SonarQube nu este accesibil, verificați DNS, TLS și ruta runnerului. Dacă SARIF nu se publică, verificați security-events write. Dacă analiza depășește timeoutul, excludeți directoarele generate și împărțiți joburile fără a reduce acoperirea critică.

Checklist pentru proiect nou

- ☐ Repository nou și depersonalizat
- ☐ Secrets configurate fără valori în Git
- ☐ Workflow manual executat cu succes
- ☐ Artefacte SARIF și SBOM disponibile
- ☐ Baseline triat și excepții cu expirare
- ☐ Job stabil devsecops-gate
- ☐ Required status check activ
- ☐ Build și deployment dependente de gate
- ☐ Imagine finală scanată
- ☐ Test negativ demonstrat
- ☐ Backup și restore SonarQube testate
- ☐ Owner și calendar de mentenanță aprobate

Anexe cu fișiere complete

Anexele reproduc fișierele distribuite cu toolkitul. Pentru producție, fixați versiunile acțiunilor și imaginilor la taguri sau digesturi aprobate.

Manual workflow

[templates/github/manual-devsecops-generic.yml](#)

```
name: Manual DevSecOps Audit

on:
  workflow_dispatch:
    inputs:
      scan_depth:
        description: Audit depth
        required: true
        default: standard
        type: choice
        options: [standard, extended]

permissions:
  contents: read

jobs:
  semgrep:
    runs-on: ubuntu-latest
    container: semgrep/semgrep:latest
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config auto --sarif --output semgrep.sarif .
        continue-on-error: true
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: semgrep-report, path: semgrep.sarif}

  trivy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scanners: vuln,secret,misconfig
          severity: HIGH,CRITICAL
          format: sarif
          output: trivy.sarif
          exit-code: '0'
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: trivy-report, path: trivy.sarif}

  gitLeaks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: {fetch-depth: 0}
      - name: GitLeaks history audit
        run: docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --source=/repo --report-format=sarif --report-path=/repo/gitleaks.sarif --exit-code=0
      - uses: actions/upload-artifact@v4
```

```
if: always()
with: {name: gitleaks-report, path: gitleaks.sarif, retention-days: 14}
```

Mandatory gate example

[templates/github/mandatory-devsecops-gate.yml.example](https://github.com/ErnuEU/mandatory-devsecops-gate.yml.example)

```
name: Mandatory DevSecOps Gate
on:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
  security-events: write
jobs:
  devsecops-gate:
    name: devsecops-gate
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: Gitleaks
        uses: gitleaks/gitleaks-action@v2
      - name: Trivy gate
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          severity: HIGH,CRITICAL
          exit-code: '1'
      - name: Semgrep gate
        run: docker run --rm -v "$PWD:/src" -w /src semgrep/semgrep:latest semgrep scan --config p/default
    --error .
```

Semgrep policy

[config/.semgrep.yml](#)

```
rules:
- id: ernu-no-shell-true-suppression
  message: Nu ascunde eșecul unui control de securitate cu `|| true`.
  severity: WARNING
  languages: [bash]
  patterns:
    - pattern-regex: '(semgrep|trivy|gitleaks).*\|\|\s*true'
- id: ernu-no-hardcoded-private-key
  message: Material de cheie privată detectat în cod.
  severity: ERROR
  languages: [generic]
  pattern-regex: '-----BEGIN (RSA |EC |OPENSSH )?PRIVATE KEY-----'
```

MegaLinter policy

[config/.mega-linter.yml](#)

```
APPLY_FIXES: none
VALIDATE_ALL_CODEBASE: true
SHOW_ELAPSED_TIME: true
PRINT_ALPACA: false
REPORT_OUTPUT_FOLDER: megalinter-reports
DISABLE_LINTERS:
  - REPOSITORY_TRIVY
  - REPOSITORY_GITLEAKS
MARKDOWN_MARKDOWNLINT_FILTER_REGEX_EXCLUDE: "(^|/)vendor/|^|/)node_modules/|^|/)artifacts/"
```

Gitleaks policy

[config/gitleaks.toml](#)

```
title = "ERNU DevSecOps Gitleaks policy"

[extend]
useDefault = true

[[allowlists]]
description = "Documented fake values and fixtures"
paths = [
    '''(^|/)\.env\.example$''',
    '''(^|/)\.fixtures?/''',
    '''(^|/)\.test-data/'''
]
regexes = [
    '''CHANGE_ME''',
    '''example\.invalid''',
    '''192\.0\.2\.[0-9]{1,3}'''
]
```

SonarQube Compose

[platform/sonarqube/docker-compose.yml](#)

```
services:
  sonarqube:
    image: sonarqube:community
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/${SONAR_DB_NAME}
      SONAR_JDBC_USERNAME: ${SONAR_DB_USER}
      SONAR_JDBC_PASSWORD: ${SONAR_DB_PASSWORD}
    ports:
      - "127.0.0.1:${SONAR_HTTP_PORT:-9000}:9000"
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
    networks: [sonar]

  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${SONAR_DB_NAME}
      POSTGRES_USER: ${SONAR_DB_USER}
      POSTGRES_PASSWORD: ${SONAR_DB_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${SONAR_DB_USER} -d ${SONAR_DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 10
    volumes:
      - sonarqube_db:/var/lib/postgresql/data
    networks: [sonar]

networks:
  sonar:
    internal: true

volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
  sonarqube_db:
```

SonarQube variables

[platform/sonarqube/.env.example](#)

```
SONAR_HTTP_PORT=9000  
SONAR_DB_NAME=sonarqube  
SONAR_DB_USER=sonarqube  
SONAR_DB_PASSWORD=CHANGE_ME_USE_A_LONG_RANDOM_PASSWORD
```

Validation script

[scripts/validate-configs.sh](#)

```
#!/usr/bin/env bash
set -Eeuo pipefail

root_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

python3 - <<'PY' "$root_dir"
import pathlib, sys
import yaml
root = pathlib.Path(sys.argv[1])
for path in root.rglob("*.yaml"):
    with path.open(encoding="utf-8") as handle:
        yaml.safe_load(handle)
    print(f"YAML OK: {path.relative_to(root)}")
PY

docker compose --env-file "$root_dir/platform/sonarqube/.env.example" \
-f "$root_dir/platform/sonarqube/docker-compose.yaml" config --quiet
echo "Compose OK"
```

Local audit script

[examples/sample-application/devsecops/scripts/run-local-audit.sh](#)

```
#!/usr/bin/env bash
set -Eeuo pipefail
repo_root="$(cd "$(dirname "${BASH_SOURCE[0]}")/../../.." && pwd)"
report_dir="${repo_root}/devsecops/reports"
mkdir -p "${report_dir}"
command -v docker >/dev/null || { echo "Docker este necesar." >&2; exit 2; }
docker run --rm -v "${repo_root}:/src" -w /src semgrep/semgrep:latest semgrep scan --config
/src/devsecops/config/.semgrep.yml --config p/default --sarif --output
/src/devsecops/reports/semgrep.sarif . || semgrep_status=$?
docker run --rm -v "${repo_root}:/src" aquasec/trivy:latest fs --scanners vuln,secret,misconfig --severity
HIGH,CRITICAL --format sarif --output /src/devsecops/reports/trivy.sarif /src
docker run --rm -v "${repo_root}:/src" zricethezav/gitleaks:latest detect --source=/src
--config=/src/devsecops/config/gitleaks.toml --report-format=sarif
--report-path=/src/devsecops/reports/gitleaks.sarif --exit-code=0
docker run --rm -v "${repo_root}:/tmp/lint" -e MEGALINTER_CONFIG=devsecops/config/.mega-linter.yml
oxsecurity/megalinter:v9
printf 'Rapoarte: %s; Semgrep status=%s\n' "${report_dir}" "${semgrep_status:-0}"
```

Project variables

[examples/sample-application/devsecops/config/project.env.example](#)

```
PROJECT_KEY=sample-application
SCAN_SEVERITY=HIGH,CRITICAL
SEMGREP_CONFIG=p/default
REPORT_DIR=devsecops/reports
SONAR_HOST_URL=https://sonarqube.example.invalid
SONAR_TOKEN=
```

Sonar project

[examples/sample-application/devsecops/sonar-project.properties](#)

```
sonar.projectKey=sample-application
sonar.projectName=Sample Application
sonar.sources=src
sonar.exclusions=**/vendor/**,**/node_modules/**,**/build/**
sonar.sourceEncoding=UTF-8
```